Open in Colab

`while` Statements

In Python `while` statement evaluates a condition then runs a block of code like `if` statement. But unlike `if` it repeats that block of code **while** the condition is equals to `True`.

In [1]:
```
name = input("enter your name: ")
while name == "":
  name = input("please enter your name: ")
```

Like `if` statements, `while` statements can also have an optional `else` statement.

In [2]:
```
name = input("enter your name: ")
while name == "":
  name = input("please enter your name: ")
else:
  print("hello", name)
```

```
hello x
```

Breaking The Loops

Sometimes you may want to exit from the loop even the condition was `True`. To do use `break`. If you break the loop then `else` statement is not evaluated.

In [3]:
```
counter = 3
name = input("enter your name: ")
while name == "":
    counter = counter - 1
    if counter == 0:
        break
    name = input(f"please enter your name: (You have {counter} tries)")
else:
    print("hello", name)
```

```
hello x
```

`continue` statement is used to go to the start of the loop without evaluating the remaining of the loop.

In [4]:
```
counter = 3
while not (name := input("enter your name: ")):
    if counter > 1:
        counter -= 1
        continue
    break
else:
    print("hello", name)
```

```
hello x
```

Here Walrus operator used for better readablity.

In [5]:
```
n = 20340
p = 1
while n > 0:
    d = n % 10
```

```
    if d != 0:
        p *= d
    n //= 10
print(p)
```

```
24
```

n	p	d
20340	1	0
2034	1	4
203	4	3
20	12	0
2	12	2
0	24	

Nested Loops

When using `break` inside a nested loop, you can break only the inner one.

In [6]:
```
while True:
  while True:
    break
  print("Only the inner loop is break.")
  break
```

```
Only the inner loop is break.
```

Exercise: Gretest Common Divisor (GCD) of Two Positive Integers

In [7]:
```
num1,num2 = 922,592
```

Euclidean algorithm for calculating the GCD of two integers uses the fact that:

$$gcd(a, b) = gcd(a - b, b) = gcd(a, b - a)$$

and that:

$$gcd(a, a) = a$$

In [8]:
```
a, b = 15, 12
print(f"GCD({a},{b})",end=' = ')

while a != b:
  if a > b:
    print(f"GCD({a-b},{b})",end=' = ')
    a = a - b
  if a < b:
    print(f"GCD({a},{b-a})",end=' = ')
    b = b - a
else:
  print(a)
```

```
GCD(15,12) = GCD(3,12) = GCD(3,9) = GCD(3,6) = GCD(3,3) = 3
```

The following table shows the history of the values of `a` and `b`:

a	b
15	12
3	12
3	9
3	6
3	3

At the last line, since `a==b` happens then the `gcd(15,12)` is found to be `3`.

You can improve this algorithm by converting `if` statements to `while` statements.

In [9]:
```
a, b = 15123, 1221
print(f"GCD({a},{b})",end=' = ')

while a != b:
  while a > b:
    a = a - b
  else:
    print(f"GCD({a},{b})",end=' = ')
  while a < b:
    b = b - a
  else:
    print(f"GCD({a},{b})",end=' = ')
else:
  print(a)
```

```
GCD(15123,1221) = GCD(471,1221) = GCD(471,279) = GCD(192,279) = GCD(192,87) = GCD(18,8
7) = GCD(18,15) = GCD(3,15) = GCD(3,3) = 3
```

a	b
15	12
3	12
3	3

Here it is calculated within less steps.

Also you can use the fact that

$$gcd(a, b) = gcd(a, b\%a)$$

and that:

$$gcd(a, 0) = a$$

Here you dont have to check if a < b or not in each step. Instead just check if $b = 0$ or not.

In [10]:
```
a, b = 15123, 1221

print(f"GCD({a},{b}) is equal to the last item in the following sequence:")

while b:
    print(b, end=' ')
    a, b = b, a % b
```

```
GCD(15123,1221) is equal to the last item in the following sequence:
1221 471 279 192 87 18 15 3
```

a	b

a	b
15123	1221
1221	471
471	279
279	192
192	87
87	18
18	15
15	3
3	0

Exercise: Fibonacci Numbers

In [11]:
```
print('Fibonacci numbers = ', end='')

a, b = 0, 1

while True:
  print(a, end=', ')
  a, b = b, a + b
  if a > 100:
    print('...')
    break
```

```
Fibonacci numbers = 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, ...
```

In [12]:
```
print('Golden numbers = ', end='')

a, b = 1, 1

while True:
  print(f"{b}/{a}={b/a}", end=', ')
  a, b = b, a + b
  if a > 100:
    print('...')
    break
```

```
Golden numbers = 1/1=1.0, 2/1=2.0, 3/2=1.5, 5/3=1.6666666666666667, 8/5=1.6, 13/8=1.62
5, 21/13=1.6153846153846154, 34/21=1.619047619047619, 55/34=1.6176470588235294, 89/55=
1.6181818181818182, 144/89=1.6179775280898876, ...
```

Exercise: 6174

In [14]:
```
a = f"{int(input()):04}"
previous = ''

while a != previous:
    previous = a
    a = "".join(sorted(a))
    a = f"{int(a[::-1]) - int(a):04}"
a
```

Out[14]:
```
'6174'
```